

# ASS Viva Prep — Labs 1 to 5

Madam will not stop at “what does this program do”. She will ask the **name of the function, which header, return type, what 0 / -1 / NULL mean, why this flag, difference between two similar commands**. Answer in one breath: *name + header + what it returns + one tiny example*.

This file follows **your actual programs**, not a generic OS textbook.

## How to answer (template)

- **Command:** what it does, one flag she might ask, difference from a cousin command.
- **System call / library function:** `#include`, prototype, success/failure, `errno/perror`.
- **Type:** size is not needed; say *signed/unsigned, used for what*.
- If you used it in a lab, say **which question**.

## Tiny things she will still ask

### Shell punctuation

| Symbol / word                             | What it is    | Exact meaning                                                        | Example from your labs                                | Viva trap                                                                              |
|-------------------------------------------|---------------|----------------------------------------------------------------------|-------------------------------------------------------|----------------------------------------------------------------------------------------|
| <code>#!/bin/bash</code>                  | shebang       | Kernel reads first line and runs <code>/bin/bash</code> on this file | every <code>.sh</code>                                | Without it, <code>./script</code> may run with <code>sh</code> , not <code>bash</code> |
| <code>\$0</code>                          | special param | Name used to invoke the script                                       | Lab 1 Q5 usage line                                   | Can be <code>./sample.sh</code> or full path                                           |
| <code>\$1 \$2 \$3</code>                  | positional    | 1st, 2nd, 3rd argument                                               | Q5: file, start, end                                  | <code>\$10</code> is <code>\$1</code> then <code>0</code> unless <code>\${10}</code>   |
| <code>\$#</code>                          | count         | Number of arguments, not including <code>\$0</code>                  | <code>[ \$# -ne 3 ]</code>                            | Empty call $\rightarrow$ <code>\$#</code> is 0                                         |
| <code>\$@</code>                          | all args      | Each argument as a <b>separate</b> word when quoted                  | Lab 2 Q3 for file in <code>"\$@"</code>               | <code>"\$*" is one string</code>                                                       |
| <code>\$?</code>                          | status        | Exit code of <b>last</b> command; 0 = success, non-zero = fail       | after <code>grep/gcc</code>                           | Check immediately; next command overwrites it                                          |
| <code>`cmd` / <code>\$(cmd)</code></code> | substitution  | Replace with stdout of <code>cmd</code>                              | <code>len=\expr ...`</code>                           | Trailing newline is stripped                                                           |
| <code>2&gt;/dev/null</code>               | redirect      | File descriptor 2 (stderr) discarded                                 | <code>factorial [ "\$n" -lt 0 ] 2&gt;/dev/null</code> | 1 is stdout, 2 is stderr, 0 is stdin                                                   |
| <code>;</code>                            | sequence      | Run next after previous finishes                                     | <code>gcc a.c ; ./a.out</code>                        | Always runs next even if first failed (&& does not)                                    |
| <code>&amp;</code>                        | background    | Start job, shell does not wait                                       | Lab 5 Q4 <code>./sample &amp;</code>                  | PID printed; use <code>wait</code> or <code>fg</code>                                  |
| <code>\ </code>                           | pipe          | stdout of left becomes stdin of right                                | <code>who \  wc -l</code>                             | Both run concurrently                                                                  |
| <code>&gt;</code>                         | redirect      | Overwrite stdout file                                                | <code>sort ... &gt; merged.txt</code>                 | File created even if empty; destroys old content                                       |
| <code>&gt;&gt;</code>                     | append        | Add to end of file                                                   | <code>echo hi &gt;&gt; log</code>                     | Creates file if missing                                                                |
| <code>[ ] / test</code>                   | builtin       | <code>[ is test; ] is required</code>                                | <code>[ ! -d "\$dirname" ]</code>                     | Spaces required: <code>[ -f f ]</code> not <code>[-f f]</code>                         |
| <code>-f</code>                           | test          | True if path is a <b>regular file</b>                                | Lab 1 Q4                                              | False for directories and missing paths                                                |
| <code>-d</code>                           | test          | True if path is a <b>directory</b>                                   | Lab 1 Q2                                              | False for a normal file                                                                |

| Symbol / word              | What it is | Exact meaning                         | Example from your labs        | Viva trap                                                    |
|----------------------------|------------|---------------------------------------|-------------------------------|--------------------------------------------------------------|
| -e                         | test       | True if path <b>exists</b> (any type) | Lab 2 Q3                      | True for file, dir, or symlink                               |
| -eq -ne -lt -le<br>-gt -ge | numeric    | Integer compare inside [ ]            | [ \$i -le \$n ]               | Do <b>not</b> use these for strings (manipal vs 10)          |
| = !=                       | string     | String equal / not equal              | [ "\$file" =<br>"\$newname" ] | In [ ] use =, not == (bash [ [ ] ] allows ==)                |
| exit 0                     | terminate  | Success to caller                     | end of happy path             | Same as return 0 in a function; in script it ends the script |
| exit 1                     | terminate  | Failure to caller                     | usage errors                  | Convention: 1 = general error                                |

## C punctuation and return values

| Token           | Type / kind         | What it holds                                     | Success / failure                            | Where you used it                |
|-----------------|---------------------|---------------------------------------------------|----------------------------------------------|----------------------------------|
| argc            | int                 | Count of argv slots <b>including</b> program name | —                                            | every C main with args           |
| argv[0]         | char *              | How the program was invoked                       | never NULL in a normal run                   | printf("Usage: %s ...", argv[0]) |
| argv[1]         | char *              | First user argument                               | missing if argc < 2                          | word, filename, n, oldpath       |
| NULL            | pointer constant    | "no object"                                       | fopen / opendir / strstr fail                | Lab 3–5                          |
| EOF             | int (–1)            | End of file / stream                              | getc loop stops                              | Lab 3 Q4                         |
| 0 from syscall  | int                 | Usually <b>success</b>                            | stat, link, unlink, chmod                    | Lab 4                            |
| –1 from syscall | int                 | <b>Failure</b> ; errno set; call perror           | all Unix calls                               | Lab 3 extra, 4, 5                |
| pid_t           | signed integer type | Process ID                                        | fork special (see Lab 5)                     | Lab 5                            |
| uid_t / gid_t   | integer             | User / group id                                   | compare with getuid()                        | Lab 3 extra, Lab 4               |
| mode_t          | integer bits        | Type + rwxrwxrwx                                  | pass to chmod                                | Lab 3 extra, Lab 4 st_mode       |
| ino_t / st_ino  | unsigned integer    | Inode number on that device                       | unique <b>per filesystem</b> (st_dev+st_ino) | Lab 4                            |
| off_t           | signed offset       | Byte position / file size                         | lseek returns –1 on error                    | Lab 3 extra                      |
| ssize_t         | signed size         | Byte count from read/write/readlink               | 0 = EOF for read; –1 = error                 | Lab 3 extra, Lab 4 Q4            |
| size_t          | unsigned size       | sizeof, strlen, buffer lengths                    | never negative                               | Lab 3 extra .c check             |
| FILE *          | pointer             | Buffered stdio stream                             | NULL if fopen fails                          | Lab 3 Q1 Q2 Q4                   |
| DIR *           | pointer             | Open directory                                    | NULL if opendir fails                        | Lab 3 extra, Lab 4 extra         |
| struct stat     | struct              | Snapshot of inode metadata                        | filled by stat/lstat/fstat                   | Lab 4                            |
| struct dirent   | struct              | One directory entry; use d_name                   | readdir returns NULL at end                  | Lab 3 extra, Lab 4 extra         |

## Headers (name + what lives inside + which lab)

| Header     | What it declares (say these names)                                                                  | Typical return                  | Labs    |
|------------|-----------------------------------------------------------------------------------------------------|---------------------------------|---------|
| <stdio.h>  | printf scanf fopen fclose fgetc getc putc getchar fflush perror snprintf NULL EOF FILE stdin stdout | FILE * or int; NULL/EOF on fail | 3, 4, 5 |
| <stdlib.h> | exit malloc free atoi NULL                                                                          | exit does not return            | 3, 5    |

| Header        | What it declares (say these names)                                                                  | Typical return                                 | Labs                          |
|---------------|-----------------------------------------------------------------------------------------------------|------------------------------------------------|-------------------------------|
| <string.h>    | strstr strlen strcmp strcpy                                                                         | strstr → pointer or NULL;<br>strcmp 0 if equal | 3 extra                       |
| <unistd.h>    | fork execl getpid getppid getuid sleep link symlink<br>readlink unlink read write lseek close chdir | see each call; many use -1                     | 3 extra, 4, 5                 |
| <sys/types.h> | pid_t uid_t off_t ssize_t                                                                           | types only                                     | 5 (often pulled in by others) |
| <sys/wait.h>  | wait waitpid WIFEXITED WEXITSTATUS WIFSIGNALED<br>WTERMSIG                                          | wait → child PID or -1                         | 5                             |
| <sys/stat.h>  | stat lstat fstat chmod struct stat S_IROTH S_ISREG                                                  | 0 / -1                                         | 3 extra, 4                    |
| <fcntl.h>     | open O_RDONLY O_WRONLY O_CREAT O_TRUNC                                                              | fd ≥ 0 or -1                                   | 3 extra Q6                    |
| <dirent.h>    | opendir readdir closedir struct dirent                                                              | DIR */dirent * or<br>NULL                      | 3 extra, 4 extra              |
| <time.h>      | ctime time_t                                                                                        | ctime → char * with \n                         | Lab 4 Q2 Q6                   |

## Lab 1 — Linux commands and tiny scripts

Q1 is **every command**. This is where “even date” comes from.

### Q1 commands (full table — this is the “even date” section)

| Command | What it does                                 | Important flags / form                                      | Sample                      | Difference / viva note                                     |
|---------|----------------------------------------------|-------------------------------------------------------------|-----------------------------|------------------------------------------------------------|
| who     | List <b>login sessions</b> (user, tty, time) | who am i current session                                    | who \  wc -l<br>Lab 1 Q3    | Not unique users; two terminals = two lines                |
| whoami  | Print <b>effective user name</b> only        | none                                                        | whoami                      | vs who: one word vs table of sessions                      |
| id      | uid, gid, groups                             | id -u numeric uid                                           | (not in Q1 but she may ask) | vs whoami: numbers + groups                                |
| echo    | Write arguments to stdout                    | -n no newline                                               | echo "hi" \$HOME            | Shell expands \$var unless quoted                          |
| ls      | List names in a directory                    | default: current dir                                        | ls /tmp                     | Does not show . files                                      |
| ls -l   | Long listing                                 | permissions, nlink, owner, group, size, <b>mtime</b> , name | Lab 2 Q4                    | First char: - file d dir l symlink                         |
| ls -a   | All names                                    | includes . hidden                                           | ls -a                       | . = this dir, .. = parent                                  |
| ls -lu  | Long listing using <b>atime</b>              | -l + -u                                                     | Lab 2 Q4 access column      | -u without -l only sorts by atime                          |
| ls -lc  | Long listing using <b>ctime</b>              | status change                                               | compare with stat           | chmod/link update ctime, not always mtime                  |
| pwd     | Print working directory                      | absolute path                                               | pwd                         | Logical path; pwd -P physical (no symlinks)                |
| cd      | Change directory                             | cd home; cd - previous; cd .. parent                        | cd lab4/q1                  | Builtin; failure if no execute permission on dir           |
| cat     | Concatenate files to stdout                  | cat f1 f2                                                   | cat notes.txt               | No arguments → reads stdin                                 |
| cp      | Copy file                                    | cp src dest; -r recursive                                   | cp a b                      | New inode (usually); dest overwritten                      |
| mv      | Rename or move                               | mv old new                                                  | Lab 2 Q3                    | Same filesystem: rename directory entry, <b>same inode</b> |
| rm      | Unlink file                                  | -i ask; -r recursive                                        | Lab 2 Q2                    | No trash. Directory needs -r (or rmdir if empty)           |
| mkdir   | Create directory                             | -p create parents                                           | mkdir testdir               | Mode often 0755 minus umask                                |

| Command  | What it does                                  | Important flags / form                         | Sample                 | Difference / viva note                                  |
|----------|-----------------------------------------------|------------------------------------------------|------------------------|---------------------------------------------------------|
| rm dir   | Remove <b>empty</b> directory                 | —                                              | rm dir testdir         | Non-empty → error; then rm -r                           |
| chmod    | Change permission bits                        | octal or u+x                                   | chmod 644 f            | See octal table below                                   |
| date     | Print or set clock                            | date +%Y-%m-%d %H:%M:%S                        | Lab 1 Q3               | This is <b>date</b> , not dt. Default = local date+time |
| clear    | Clear terminal screen                         | —                                              | —                      | Sends ANSI escape; history still in scrollbar           |
| wc       | Count lines/words/bytes                       | -l -w -c                                       | who   wc -l            | -c bytes; -m characters                                 |
| touch    | Create empty file <b>or</b> update timestamps | -a atime only -m mtime only                    | touch f                | Existing file: mtime+atime now                          |
| grep     | Print lines matching pattern                  | -n -w -i -v                                    | Lab 1 Q4 -n -w manipal | -w whole word; regex engine (unlike strstr)             |
| sort     | Sort lines                                    | -n numeric -u unique -r reverse                | Lab 2 extra            | Default is <b>lexicographic</b> (10 before 2)           |
| sed      | Stream editor                                 | -n '5,10p' print range                         | Lab 1 Q5               | -n suppress default print; p print                      |
| ps       | Process snapshot                              | ps yours; ps -e/aux all; ps -l long            | Lab 5 Q4               | STAT R run S sleep Z zombie T stopped                   |
| sleep    | Pause                                         | argument = <b>seconds</b>                      | Lab 5 Q1 sleep(2) in C | Shell: sleep 5. C: unsigned sleep(unsigned)             |
| kill     | Send a signal                                 | default SIGTERM 15; -9 SIGKILL                 | kill PID               | Argument is <b>PID</b> , not name. -9 cannot be caught  |
| expr     | Evaluate expression                           | length: expr "\$s" : '.*' ;<br>expr \$a \* \$b | Lab 2 Q1 Q5            | * must be \* or glob expands                            |
| tr       | Translate characters                          | tr 'a-z' 'A-Z'                                 | Lab 2 Q3               | Reads stdin; we piped echo "\$file"                     |
| basename | Strip directory                               | basename /a/b/c → c                            | Lab 2 Q2               | Opposite is dirname                                     |
| awk      | Column processor                              | {print \$1} \$5 \$6,\$7,\$8                    | Lab 2 Q4               | \$1 first field of <b>ls -l</b> line                    |
| bc       | Calculator                                    | -l math lib; scale=4                           | Lab 2 extra quadratic  | Needed for floats; expr is integers                     |

## chmod octal (she will ask)

| Octal digit | Binary | Letters | Meaning                 |
|-------------|--------|---------|-------------------------|
| 0           | 000    | ---     | no permission           |
| 1           | 001    | --x     | execute / search (dirs) |
| 2           | 010    | -w-     | write                   |
| 3           | 011    | -wx     | write + execute         |
| 4           | 100    | r--     | read                    |
| 5           | 101    | r-x     | read + execute          |
| 6           | 110    | rw-     | read + write            |
| 7           | 111    | rwX     | all                     |

| Mode | Letters    | Typical use                                    |
|------|------------|------------------------------------------------|
| 644  | rw-r--r--  | ordinary file (Lab 3 extra 0644 for new files) |
| 755  | rxwxr-xr-x | executable / directory                         |
| 600  | rw-----    | private file                                   |
| 777  | rxwxrxwx   | never recommend in viva                        |

| Mode         | Letters     | Typical use                     |
|--------------|-------------|---------------------------------|
| 0004 S_IROTH | others read | Lab 3 extra st_mode \   S_IROTH |

| Who          | Bit names               | Octal place |
|--------------|-------------------------|-------------|
| owner (user) | S_IRUSR S_IWUSR S_IXUSR | hundreds    |
| group        | S_IRGRP S_IWGRP S_IXGRP | tens        |
| others       | S_IROTH S_IWOTH S_IXOTH | units       |

Directory: **r** = list names, **x** = cd / traverse, **w** = create/delete names.

## ls -l permission string

| Position | Meaning    | Examples                                         |
|----------|------------|--------------------------------------------------|
| 1        | file type  | – regular, d directory, l symlink, c char device |
| 2–4      | owner rwx  | rwX or rw–                                       |
| 5–7      | group rwx  | r–X                                              |
| 8–10     | others rwx | r––                                              |

Example: –rw–r–r– = regular file, 644.

## Identity and time commands

| Command | Prints           | Source        | Lab     |
|---------|------------------|---------------|---------|
| date    | calendar + clock | system clock  | 1 Q3    |
| who     | every login      | utmp          | 1 Q3    |
| whoami  | one name         | effective uid | 1 Q1    |
| id      | uid gid groups   | same + extra  | compare |
| pwd     | current dir      | process cwd   | 1 Q1    |

## grep vs Lab 3 C grep

|                  | Shell grep                     | Your strstr grep       |
|------------------|--------------------------------|------------------------|
| Match            | regex + optional –w word       | substring anywhere     |
| "man" vs manipal | –w → no; default regex may yes | <b>yes</b> (substring) |
| Line numbers     | –n                             | you did not print them |
| Binary           | real UNIX utility              | teaching subset        |

## Lab 1 scripts

**Q2** — read dirname, [ ! -d ], ls "\$dirname"/\*.c

- -d true if directory.

- Glob \*.c expands to matching names; if none, bash may leave the glob or error depending on nullglob.

- Quote "\$dirname" so spaces in path don't break.

**Q3** — date then who | wc -l

- Pipe: who lines → wc -l counts them = number of login sessions (one user can count twice if two terminals).

**Q4** — grep -n -w "manipal" file

- -w whole word: manipal matches, manipalxyz does not.

- -n prints line:the line.

- [ ! -f ] regular file (not directory).

**Q5** — arguments \$1 file, \$2 start, \$3 end. `sed -n 's,ep'`  
- \$# -ne 3 means not exactly 3 args.  
- `sed -n "5,10p"` print lines 5 through 10 only (`-n` = no default print, `p` = print).  
- head/tail alternative; you used sed.

---

## Lab 2 — Shell scripts

### Q1 — length ≥ 10 with `case` and `expr`

```
len=`expr "$str" : '.*'`  
case $len in  
[0-9]) ... too short ;;  
*)      ... ok ;;  
esac
```

- `expr STRING : REGEXP` returns length of match. `.*` matches whole string → length.
- `expr` is **external**; `$( ( ))` is bash arithmetic (newer).
- `case` matches **patterns**, not numbers as C would.
- `[0-9]` is **one digit** 0–9, so lengths 0–9. Length 10+ falls in `*`.
- `;;` ends a case arm. `esac` ends case.
- `read str` reads one line from stdin into `str`.

**Why not `[ $len -lt 10 ]`?** Question forced `case` and `expr`.

### Q2 — delete files in d2 that also exist in d1 (by name, not content)

- `$1 d2=$2`
- `for file in "$d1"/*`
- `basename` strips directory → `notes.txt`
- `[ -f "$d2/$fname" ] then rm`
- Identical **name**, not `cmp/diff`. She may trick you: same name different content → still deleted.

### Q3 — rename to uppercase if free

- `"$@"` each argument
- `[ ! -e ]` does not exist
- `tr 'a-z' 'A-Z'` transliterate
- `mv` rename
- `[ "$file" -ef "$newname" ]` same file (macOS case-insensitive disk). On Linux usually not needed.

### Q4 — formatted listing

From `ls -l`: field 1 permissions, 5 size, 6–8 mtime.

`ls -lu`: those date fields become **atime**.

`awk '{print $1}'` first column.

`printf "%-12s"` left pad width 12.

`basename filename` only.

## mtime vs atime vs ctime

| Field | Full name         | Changes when                             | See it with                   | Does touch change it?    |
|-------|-------------------|------------------------------------------|-------------------------------|--------------------------|
| mtime | modification time | file <b>contents</b> written             | <code>ls -l, st_mtime</code>  | yes (default)            |
| atime | access time       | file <b>read</b> (cat, grep)             | <code>ls -lu, st_atime</code> | yes (default)            |
| ctime | change time       | <b>inode</b> changes: chmod, chown, link | <code>ls -lc, st_ctime</code> | usually as a side effect |

chmod updates **ctime**, not mtime.

## Q5 — factorial

- `while [ $i -le $n ]`
- `expr $fact \* $i` — `*` must be escaped or shell glob-expands.
- `expr` integer only; big `n` overflows shell arithmetic.
- Negative rejected with `[ "$n" -lt 0 ]`.

## Extra Q6 — merge sorted unique

```
sort -n -u file1 file2 > out
- -n numeric (10 after 9, not after 1)
- -u unique
- ${3:-merged.txt} default outfile if $3 empty
```

## Extra Q7 — quadratic with `case`

- Discriminant  $b^2-4ac$  via `bc` (floating point; `expr` cannot do floats well)
- `case $kind in positive|zero|negative)`
- `scale=4` decimal places in `bc`
- `a=0` is not quadratic

---

# Lab 3 — C / stdio / UNIX utilities

## argc / argv (every C lab)

```
./sample manipal data.txt
argv[0] = "./sample"
argv[1] = "manipal"
argv[2] = "data.txt"
argc = 3
```

## Q1 — mini grep

- `fopen(path, "r")` → `FILE *` or `NULL`
- `fgets(buf, n, fp)` reads one line including `'\n'`, `NULL` at EOF
- `strstr(line, word)` pointer to first substring or `NULL`
- `fclose`
- Not regex. "man" matches manipal. Real `grep -w` would not.

**fopen modes:** "r" read, "w" write (truncate/create), "a" append, "r+" read/write.

## Q2 — mini more

- Loop fgets, count lines, every 20 print --More--, fflush(stdout) then getchar()
- **Why fflush?** Prompt must appear before waiting. Buffered stdout.
- getchar() returns int (not char) so EOF fits.
- Multiple files: for (i = 1; i < argc; i++)

## Q3 — printf conversion specifiers

Say the letter **and** the C type:

| Specifier | Expected argument | Prints              | Example in Q3 (n=255, d=123.456) | Notes                                            |
|-----------|-------------------|---------------------|----------------------------------|--------------------------------------------------|
| %d        | int               | signed decimal      | 255                              | most common integer                              |
| %i        | int               | signed decimal      | 255                              | same as %d in <b>printf</b> (different in scanf) |
| %u        | unsigned int      | unsigned decimal    | 4000000000 with u suffix         | do not pass a negative int                       |
| %o        | unsigned int      | octal, no leading 0 | 377                              | 255 = 3×64+7×8+7                                 |
| %x        | unsigned int      | hex lowercase       | ff                               |                                                  |
| %X        | unsigned int      | hex uppercase       | FF                               |                                                  |
| %f        | double            | fixed decimal       | 123.456000                       | float is promoted to double                      |
| %e        | double            | scientific          | 1.234560e+02                     |                                                  |
| %E        | double            | scientific upper E  | 1.234560E+02                     |                                                  |
| %g        | double            | shorter of %f/%e    | 123.456                          | trailing zeros dropped                           |
| %G        | double            | like %g with E      |                                  |                                                  |
| %c        | int (character)   | one character       | A                                |                                                  |
| %s        | char *            | string until '\0'   | Manipal                          | never pass NULL                                  |
| %p        | void *            | pointer             | 0x7f...                          | cast (void *)str                                 |
| %%        | none              | a percent sign      | %                                |                                                  |

| Flag / width | Form   | Effect                                         | Q3 example       |
|--------------|--------|------------------------------------------------|------------------|
| width        | %10d   | at least 10 columns, pad spaces on <b>left</b> | [ 255]           |
| -            | %-10d  | pad on <b>right</b> (left-justify)             | [255 ]           |
| 0            | %08d   | pad with zeros                                 | [00000255]       |
| +            | %+d    | always show sign                               | [+255]           |
| precision    | %.2f   | 2 digits after decimal                         | 123.46 (rounded) |
| both         | %10.2f | width 10 and 2 decimals                        | [ 123.46]        |

printf returns the number of characters printed, or negative on error. Your program ignores it.

## Q4 — character copy

- getc(src) / putc(ch, dest) from stdio.h
- ch is int, not char, because EOF is -1; if char is unsigned, loop never ends.
- fopen dest "w"

- Count characters until EOF
- Difference `getc` vs `fgetc`: same for practical purposes; `getc` may be a macro.

## stdio vs Unix I/O

| Topic       | stdio (Lab 3 Q1–Q4)                                          | Unix I/O (Lab 3 extra Q6)                                                                   |
|-------------|--------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| Handle type | <code>FILE *</code>                                          | <code>int fd</code> (0 <code>stdin</code> , 1 <code>stdout</code> , 2 <code>stderr</code> ) |
| Open        | <code>fopen(path, "r"/"w")</code>                            | <code>open(path, O_RDONLY)</code> or <code>O_WRONLY   O_CREAT   O_TRUNC</code>              |
| Open fail   | <code>NULL</code>                                            | <code>-1</code>                                                                             |
| Read        | <code>fgets</code> , <code>getc</code> , <code>fread</code>  | <code>read(fd, buf, n) → ssize_t</code>                                                     |
| Write       | <code>putc</code> , <code>fputs</code> , <code>fwrite</code> | <code>write(fd, buf, n)</code>                                                              |
| Position    | <code>fseek</code>                                           | <code>lseek</code>                                                                          |
| Close       | <code>fclose</code>                                          | <code>close</code>                                                                          |
| Buffering   | yes                                                          | no (unless you add it)                                                                      |
| Header      | <code>stdio.h</code>                                         | <code>fcntl.h + unistd.h</code>                                                             |

Lab 3 Q4 = `stdio`. Lab 3 extra `lseek` = Unix `fds`.

## Extra Q5 — `chmod` others-read on your `.c` files

- `opendir / readdir / closedir`
- `getuid()` my uid; skip files with `st.st_uid != me`
- name ends with `.c`: `strcmp(name + len - 2, ".c")`
- `chmod(path, st.st_mode | S_IROTH)` **OR** in the others-read bit, keep other bits
- `S_IROTH` is `0004`. `S_IWOTH` write others, `S_IXOTH` execute others. Owner read `S_IRUSR`.

## Extra Q6 — `lseek` thirds

- `open(..., O_RDONLY)` returns `fd ≥ 0`, or `-1`
- `O_WRONLY | O_CREAT | O_TRUNC`, mode `0644`
- `fstat(fd, &st)` size in `st.st_size (off_t)`
- `lseek(fd, offset, whence)`
- `SEEK_SET` from start
- `SEEK_CUR` from current
- `SEEK_END` from end (offset can be negative)
- Returns new offset or `(off_t)-1`
- `read/write` return `ssize_t` (bytes, 0 EOF, `-1` error)
- Split: first third, next third, remainder (so 10 bytes → 3, 3, 4)

---

## Lab 4 — inode, stat, hard/soft links

## What is an inode?

Index node: metadata **on disk**, not the name. Contains type, permissions, uid/gid, size, timestamps, link count, pointers to data blocks. **Filename lives in the directory**, pointing to the inode number.

Two names, **same inode** → hard links.

Soft link = **another inode** whose data is a path string.

## Q1 — print inode

```
#include <sys/stat.h>
int stat(const char *path, struct stat *buf);
```

- 0 success, -1 fail (ENOENT no file).
- Print `st.st_ino`.
- `stat` **follows** symlinks. `lstat` does **not** (info about the link itself). Q4 uses `lstat` on the `.soft` file.

## Q2 — every `struct stat` field (memorise names)

| Field                   | C type (typical)       | Meaning                              | What you print                           | Viva extra                                                |
|-------------------------|------------------------|--------------------------------------|------------------------------------------|-----------------------------------------------------------|
| <code>st_dev</code>     | <code>dev_t</code>     | device id of the <b>filesystem</b>   | <code>%llu</code>                        | same for all files on that volume                         |
| <code>st_ino</code>     | <code>ino_t</code>     | inode number                         | <code>%llu</code> Lab 4 Q1               | unique together with <code>st_dev</code>                  |
| <code>st_mode</code>    | <code>mode_t</code>    | type in high bits + <code>rwX</code> | <code>%o</code> e.g. <code>100644</code> | <code>100</code> = regular file, <code>644</code> = perms |
| <code>st_nlink</code>   | <code>nlink_t</code>   | number of <b>hard</b> links          | <code>%llu</code>                        | soft links do not increase this on the target             |
| <code>st_uid</code>     | <code>uid_t</code>     | owner user id                        | <code>%u</code>                          | numeric; <code>501</code> on your Mac sample              |
| <code>st_gid</code>     | <code>gid_t</code>     | owner group id                       | <code>%u</code>                          | <code>20</code> = staff on macOS often                    |
| <code>st_rdev</code>    | <code>dev_t</code>     | device number if special file        | <code>%llu</code>                        | <code>0</code> for a normal file                          |
| <code>st_size</code>    | <code>off_t</code>     | size in bytes                        | <code>%lld</code>                        | symlink: length of the <b>path string</b>                 |
| <code>st_blksize</code> | <code>blksize_t</code> | preferred I/O block                  | <code>%ld</code>                         | often <code>4096</code>                                   |
| <code>st_blocks</code>  | <code>blkcnt_t</code>  | allocated 512-byte blocks            | <code>%lld</code>                        | rounding can look “too big”                               |
| <code>st_atime</code>   | <code>time_t</code>    | last access                          | <code>%ld + ctime()</code>               | Lab 2 <code>ls -lu</code>                                 |
| <code>st_mtime</code>   | <code>time_t</code>    | last content modify                  | <code>%ld + ctime()</code>               | Lab 2 <code>ls -l</code>                                  |
| <code>st_ctime</code>   | <code>time_t</code>    | last inode/status change             | <code>%ld + ctime()</code>               | <code>chmod</code> , <code>link</code>                    |

`ctime(&st.st_atime)` from `<time.h>` returns a string that **already ends with newline**.

`st_mode` value `100644` octal: `S_IFREG + 0644`. Directory example `040755` (`S_IFDIR`).

| Macro                                | True when                         |
|--------------------------------------|-----------------------------------|
| <code>S_ISREG(m)</code>              | regular file                      |
| <code>S_ISDIR(m)</code>              | directory                         |
| <code>S_ISLNK(m)</code>              | symlink (use <code>lstat</code> ) |
| <code>S_ISCHR(m) / S_ISBLK(m)</code> | device files                      |

## Q3 — hard link

```
int link(const char *oldpath, const char *newpath); // unistd.h
int unlink(const char *path);
```

- New directory entry, **same inode**, `st_nlink` becomes 2.
- After `unlink` of the new name, `nlink` back to 1; **data stays** while `nlink > 0`.
- When `nlink` hits 0 **and** no process has the file open, blocks are freed.
- Hard link **cannot** normally cross filesystems; **cannot** link directories (except root).
- `ln old new` in shell.

## Q4 — soft / symbolic link

```
int symlink(const char *target, const char *linkpath);
ssize_t readlink(const char *path, char *buf, size_t n);
```

- New inode. `readlink` copies the path text (`original.txt`, size 12).
- `lstat` on link: different inode from target.
- `stat` on link: **follows** to target (unless broken).
- Soft link **can** cross devices and point to directories.
- Broken link: target missing; `stat` fails, `lstat` still works.
- `ln -s target link` in shell.
- `unlink` removes the link, not the target.

### Hard vs soft (say this table)

| Question                                | Hard link ( <code>link</code> / <code>ln</code> )  | Soft link ( <code>symlink</code> / <code>ln -s</code> )                 |
|-----------------------------------------|----------------------------------------------------|-------------------------------------------------------------------------|
| New inode?                              | <b>No</b> — extra name for same inode              | <b>Yes</b> — link has its own inode                                     |
| <code>st_ino</code> of name vs original | <b>equal</b>                                       | <b>different</b>                                                        |
| Target <code>st_nlink</code>            | increases by 1                                     | unchanged                                                               |
| <code>st_size</code> of the new name    | same as file data size                             | length of path ( <code>original.txt</code> → 12)                        |
| Cross filesystem?                       | <b>No</b>                                          | <b>Yes</b>                                                              |
| Link to a directory?                    | <b>No</b> (normal users)                           | <b>Yes</b>                                                              |
| Original name deleted                   | data remains ( <code>nlink</code> still $\geq 1$ ) | link <b>breaks</b> ( <code>stat</code> fails, <code>lstat</code> works) |
| Followed by <code>stat</code> ?         | N/A (it is the file)                               | <code>stat</code> follows; <code>lstat</code> does not                  |
| Shell                                   | <code>ln old new</code>                            | <code>ln -s target linkname</code>                                      |
| Your program                            | Lab 4 Q3 <code>original.txt.hard</code>            | Lab 4 Q4 <code>original.txt.soft</code>                                 |

## Extra Q5 / Q6 — directory walk

- `opendir` `readdir` skip `.` and `..`
- Build path with `sprintf(path, sizeof path, "%s/%s", dir, name)`
- Q5: print `st_ino` and `name`
- Q6: same `print_stat` as Q2 for every file
- `d_name` is only the name, not full path → you must join.

## Lab 5 — processes: fork, wait, exec, zombie, orphan

## Process vs program vs thread

- **Program:** file on disk (a.out).
- **Process:** running instance (PID, address space, open files).
- **Thread (Lab 6):** same address space. Lab 5 is **processes**.

## fork() — the exam favourite

```
#include <unistd.h>
pid_t fork(void);
```

### Three returns:

| fork() return | Process     | What the value is             | What you should do          |
|---------------|-------------|-------------------------------|-----------------------------|
| > 0           | parent      | child's PID                   | wait that pid or wait(NULL) |
| 0             | child       | not a PID; just the marker    | optionally exec, then exit  |
| -1            | parent only | error; <b>no child exists</b> | perror("fork"); return      |

There is **no** child on failure.

| Call      | Header   | Returns                         | Blocks?        |
|-----------|----------|---------------------------------|----------------|
| getpid()  | unistd.h | my PID                          | no             |
| getppid() | unistd.h | parent PID (1 if orphaned)      | no             |
| sleep(n)  | unistd.h | leftover seconds if interrupted | yes, n seconds |

After success, **two** processes continue at the next statement. Memory is copy-on-write. File descriptors are duplicated.

## wait / waitpid

```
#include <sys/wait.h>
pid_t wait(int *status);          /* any child */
pid_t waitpid(pid_t pid, int *status, int options);
```

- Parent **blocks** until a child exits (unless WNOHANG).
- Returns PID of the reaped child, or -1.
- wait(NULL) discard status.
- wait(&status) fill status.

### Status macros (Lab 5 extra Q6):

| Macro               | Header     | True / value                         | When to use                           |
|---------------------|------------|--------------------------------------|---------------------------------------|
| WIFEXITED(status)   | sys/wait.h | non-zero if child called exit/return | check before reading exit code        |
| WEXITSTATUS(status) | same       | 0–255 exit code                      | only if WIFEXITED                     |
| WIFSIGNALED(status) | same       | killed by signal                     | kill -9                               |
| WTERMSIG(status)    | same       | signal number                        | 9 = SIGKILL                           |
| WIFSTOPPED(status)  | same       | stopped (Ctrl+Z)                     | job control                           |
| status >> 8         | —          | same as WEXITSTATUS if normal exit   | exit(42) → 42; raw status often 10752 |

| wait form                      | Meaning                                               |
|--------------------------------|-------------------------------------------------------|
| wait(NULL)                     | block until <b>any</b> child exits; throw status away |
| wait(&status)                  | same, but fill status                                 |
| waitpid(pid, &status, 0)       | wait for <b>that</b> child                            |
| waitpid(pid, &status, WNOHANG) | do not block; 0 if still running                      |

exit(n) (stdlib.h) ends **this** process. \_exit skips stdio flush. return from main = exit.

## Q1 — parent waits

Child sleep(2) exit(0). Parent prints then wait(&status) then continues. Without wait, parent could finish first.

## Q2 — exec

```
int execl(const char *path, const char *arg0, ... /*, NULL */);
```

- **Replaces** the child's memory with a new program. Same PID.
- Returns **only on failure** (-1). Success = no return.
- Last argument **must be NULL**.
- arg0 is what the new program sees as argv[0] (often the name).
- You load ./program1 = compiled Q1.

**exec family (she will ask the letters):**

| Function | Path                        | Arguments                   | Environment     | Notes                                                 |
|----------|-----------------------------|-----------------------------|-----------------|-------------------------------------------------------|
| execl    | full/relative path you give | list: arg0, arg1, ..., NULL | inherits        | <b>Lab 5 Q2</b> execl("./program1", "program1", NULL) |
| execlp   | searches \$PATH if no /     | list + NULL                 | inherits        | p = PATH                                              |
| execv    | path                        | char *argv[], last NULL     | inherits        | v = vector                                            |
| execvp   | PATH                        | vector                      | inherits        | common for shells                                     |
| execle   | path                        | list + NULL                 | you pass envp[] | e = environment                                       |
| execve   | path                        | vector                      | envp            | the <b>syscall</b> ; others wrap this                 |

Letters: list, vector, path search, env. Success → **does not return**. Failure → -1, perror. Last list arg **must be NULL**. Same PID after exec; memory image replaced.

## Q3 — print PID / PPID / fork return

Child: getpid, getppid (= parent), fork return **0**.

Parent: getpid, getppid (= shell), fork return **child pid**.

You wait(NULL) so prints don't interleave.

## Q4 — zombie

- Child exit immediately. Parent sleep(15) **without wait**.
- Child is dead but has a **PCB slot** holding exit status until parent waits. ps STAT Z, command <defunct>.
- When parent later exits without wait, **init (PID 1)** adopts and reaps.

- Why exist: so parent can still `wait` and read the exit code.
- Too many zombies → PID table full (bad).

Run: `./sample` & then `ps -l` or `ps aux | grep Z`.

|               | Zombie (Q4)                                              | Orphan (extra Q5)               |
|---------------|----------------------------------------------------------|---------------------------------|
| Child         | <b>dead</b> (exit already)                               | <b>alive</b> (sleep)            |
| Parent        | <b>alive</b> , not calling <code>wait</code>             | <b>dead</b> (exit first)        |
| ps STAT       | Z <defunct>                                              | normal (S while sleeping)       |
| PPID of child | still the sleeping parent                                | becomes <b>1</b> (init/launchd) |
| Why           | keep exit status until <code>wait</code>                 | child must have a parent        |
| Cleanup       | parent <code>wait</code> , or parent dies and init reaps | init is already the parent      |

## Extra Q5 — orphan

- Parent exit immediately; child `sleep(5)`.
- Child still running, parent dead → **orphan**.
- PPID **before** = real parent. **After** `sleep` = **1** (init / macOS launchd).
- Opposite of zombie: zombie = child dead parent alive; orphan = parent dead child alive.

## Extra Q6 — `wait(&status)`

Child `exit(42)`. Parent `WIFEXITED` then `WEXITSTATUS`. Also `status >> 8`.

# Data types cheat sheet (whole ASS 1–5)

| Type                          | Signed?        | Typical use                               | Header / comes from      | Lab        | Failure / sentinel                                          |
|-------------------------------|----------------|-------------------------------------------|--------------------------|------------|-------------------------------------------------------------|
| <code>int</code>              | signed         | counts, status, loop indices              | built-in                 | all C      | <code>getc</code> must be <code>int</code>                  |
| <code>char / char[]</code>    | implementation | letters, line buffers                     | built-in                 | 3          | <code>'\0'</code> ends a string                             |
| <code>char *</code>           | pointer        | C strings, <code>argv</code> , paths      | —                        | 3–5        | <code>NULL</code> = missing                                 |
| <code>unsigned int</code>     | unsigned       | <code>%u</code> values                    | built-in                 | 3 Q3       | wrap on overflow                                            |
| <code>double</code>           | floating       | <code>%f %e %g</code>                     | built-in                 | 3 Q3       | <code>printf</code> wants double                            |
| <code>long / long long</code> | signed         | printing sizes/inodes                     | built-in                 | 4          | cast <code>st_ino</code> to unsigned <code>long long</code> |
| <code>pid_t</code>            | signed         | PIDs                                      | <code>sys/types.h</code> | 5          | <code>fork -1</code> error, 0 child                         |
| <code>uid_t</code>            | integer        | user id                                   | <code>sys/types.h</code> | 3 extra, 4 | <code>getuid()</code>                                       |
| <code>gid_t</code>            | integer        | group id                                  | <code>sys/types.h</code> | 4          | <code>st_gid</code>                                         |
| <code>mode_t</code>           | bits           | permissions                               | <code>sys/stat.h</code>  | 3 extra, 4 | <code>0644</code>                                           |
| <code>off_t</code>            | signed         | file offset, <code>st_size</code>         | <code>sys/types.h</code> | 3 extra    | <code>lseek</code> → <code>(off_t)-1</code>                 |
| <code>ssize_t</code>          | signed         | read/write/readlink count                 | <code>unistd.h</code>    | 3 extra, 4 | <code>-1</code> error; <code>0</code> EOF on read           |
| <code>size_t</code>           | unsigned       | <code>sizeof</code> , <code>strlen</code> | <code>stddef.h</code>    | 3 extra    | never <code>-1</code>                                       |
| <code>time_t</code>           | integer        | Unix time seconds                         | <code>time.h</code>      | 4          | pass address to <code>ctime</code>                          |

| Type          | Signed? | Typical use        | Header / comes from | Lab        | Failure / sentinel        |
|---------------|---------|--------------------|---------------------|------------|---------------------------|
| FILE *        | pointer | stdio stream       | stdio.h             | 3          | NULL                      |
| DIR *         | pointer | directory stream   | dirent.h            | extras     | NULL                      |
| struct stat   | struct  | inode snapshot     | sys/stat.h          | 3 extra, 4 | fill via stat             |
| struct dirent | struct  | one readdir record | dirent.h            | extras     | d_name is not a full path |
| void *        | pointer | %p                 | —                   | 3 Q3       | cast required for %p      |

int main(void) — no args. int main(int argc, char \*argv[]) — command-line input.

---

## Signals and `kill` (Lab 1, useful in Lab 5 talk)

| Signal  | Number | How you send it          | Default action     | Catchable? |
|---------|--------|--------------------------|--------------------|------------|
| SIGTERM | 15     | kill PID                 | terminate          | yes        |
| SIGKILL | 9      | kill -9 PID              | terminate          | <b>no</b>  |
| SIGINT  | 2      | Ctrl+C                   | terminate          | yes        |
| SIGTSTP | 20/18  | Ctrl+Z                   | stop (job control) | yes        |
| SIGCHLD | 17     | kernel, when child exits | ignore / wait      | —          |

A **zombie** is already dead; `kill -9` on it does nothing useful. Reap with `wait`, or kill the **parent**.

---

## Rapid-fire (practise out loud)

1. who vs whoami?
2. `ls -l` vs `ls -a` vs `ls -lu`?
3. `date` what does it print? format `date +%H:%M:%S`?
4. `chmod 755` bits?
5. `rmdir` on non-empty?
6. `$$` `$0` `$1`?
7. `[ -f ]` vs `[ -d ]` vs `[ -e ]`?
8. Why `expr $a \* $b` backslash?
9. case ends with? (`esac`)
10. `grep -w` vs `strstr`?
11. Why is `getc` assigned to `int`?
12. `fopen` failure value?
13. `%d` vs `%u` vs `%x` vs `%s` vs `%p`?
14. `SEEK_SET` `SEEK_CUR` `SEEK_END`?
15. inode stores name? (**no**)
16. `stat` vs `lstat`?
17. hard link `nlink`? same inode?

18. soft link own inode? readlink?
19. unlink deletes data always? (**only if last name and not open**)
20. fork three returns?
21. exec return on success? (**never**)
22. execl last argument? (**NULL**)
23. zombie vs orphan?
24. WEXITSTATUS which byte?
25. PID of init? (**1**)
26. wait(NULL) vs wait(&status)?
27. Why fflush before getchar in more?
28. ps STAT Z?
29. sleep argument unit? (**seconds**)
30. wc -l on who counts what?

If you can answer these without looking, you are ready for Labs 1–5.

## What lives in *your* folders (so you don't freeze)

| Lab | Q     | File         | Language | What it does               | Commands / calls to name in viva  | How you run it           |
|-----|-------|--------------|----------|----------------------------|-----------------------------------|--------------------------|
| 1   | 1     | (commands)   | shell    | all listed UNIX commands   | who date ls -l chmod grep ps kill | type them                |
| 1   | 2     | sample.sh    | bash     | list *.c in a directory    | read, [ -d ], ls dir/*.c          | ./sample.sh then dirname |
| 1   | 3     | sample.sh    | bash     | date/time + login count    | date, who \   wc -l               | ./sample.sh              |
| 1   | 4     | sample.sh    | bash     | lines with word manipal    | [ -f ], grep -n -w                | enter filename           |
| 1   | 5     | sample.sh    | bash     | print line range           | \$/, sed -n 's,e p'               | ./sample.sh file 2 5     |
| 2   | 1     | sample.sh    | bash     | string length $\geq 10$    | expr : '.*', case esac            | stdin string             |
| 2   | 2     | sample.sh    | bash     | delete name clashes in d2  | basename, [ -f ], rm              | ./sample.sh d1 d2        |
| 2   | 3     | sample.sh    | bash     | rename to UPPER            | tr, mv, -e -ef                    | ./sample.sh hello.txt    |
| 2   | 4     | sample.sh    | bash     | perm size name mtime atime | ls -l, ls -lu, awk, printf        | ./sample.sh notes.txt    |
| 2   | 5     | sample.sh    | bash     | n!                         | while, expr \*                    | enter n                  |
| 2   | extra | q6/sample.sh | bash     | merge unique numeric       | sort -n -u                        | two files                |
| 2   | extra | q7/sample.sh | bash     | quadratic roots            | bc, case on discriminant          | a b c                    |
| 3   | 1     | sample.c     | C        | mini grep                  | fopen fgets strstr fclose         | ./sample word file       |
| 3   | 2     | sample.c     | C        | mini more                  | fgets, fflush, getchar            | ./sample longfile.txt    |
| 3   | 3     | sample.c     | C        | printf specifiers          | %d %x %f %s %p flags              | ./sample                 |
| 3   | 4     | sample.c     | C        | char copy                  | getc putc int ch until EOF        | ./sample src dest        |
| 3   | extra | q5/sample.c  | C        | others-read on your .c     | opendir getuid chmod S_IROTH      | ./sample [dir]           |

| Lab | Q     | File        | Language | What it does           | Commands / calls to name in viva | How you run it               |
|-----|-------|-------------|----------|------------------------|----------------------------------|------------------------------|
| 3   | extra | q6/sample.c | C        | copy file thirds       | open lseek read write            | ./sample file                |
| 4   | 1     | sample.c    | C        | print inode            | stat, st_ino                     | ./sample notes.txt           |
| 4   | 2     | sample.c    | C        | dump struct stat       | every st_* + ctime               | ./sample notes.txt           |
| 4   | 3     | sample.c    | C        | hard link then unlink  | link unlink st_nlink             | ./sample original.txt        |
| 4   | 4     | sample.c    | C        | soft link then unlink  | symlink readlink lstat           | ./sample original.txt        |
| 4   | extra | q5/sample.c | C        | inodes in a directory  | readdir stat                     | ./sample ../q1               |
| 4   | extra | q6/sample.c | C        | stat every file in dir | same + print_stat                | ./sample ../q1               |
| 5   | 1     | sample.c    | C        | parent waits           | fork wait sleep                  | ./sample                     |
| 5   | 2     | sample.c    | C        | child exec Q1          | execl("./program1", ...)         | compile Q1 as program1 first |
| 5   | 3     | sample.c    | C        | PID PPID fork-return   | getpid getppid                   | ./sample                     |
| 5   | 4     | sample.c    | C        | zombie                 | child exit, parent sleep no wait | ./sample & then ps -l        |
| 5   | extra | q5/sample.c | C        | orphan                 | parent exit, child sleep, PPID→1 | ./sample                     |
| 5   | extra | q6/sample.c | C        | exit code 42           | wait(&status) WEXITSTATUS        | ./sample                     |